

Министерство науки и высшего образования Российской

Федерации

Федеральное государственное автономное образовательное

учреждение

высшего образования «Национальный исследовательский

университет

«Московский институт электронной техники»

Лабораторная работа №7

по дисциплине «Управление качеством программного

обеспечения»

«Метрики Лоренца и Кидда»

Москва, 2024

Для успешной сдачи лабораторной работы необходимо выполнить одно задание по варианту.

На 4:

(«Номер компьютера» - 1) % 4 + 1 = **Номер задания**

Первый – Задание №1, Второй – Задание №2 и т.д.

Пятый – Задание №1, Шестой – Задание №2 и т.д.

Таким образом задание будет в диапазоне от №1 до №4

На максимальный балл:

Первый – Задание №5, Второй – Задание №6,

Третий – Задание №7, Четвертый – Задание №9

Пятый – Задание №5, Шестой – Задание №6,

Седьмой – Задание №7, Восьмой – Задание №9 и т.д.

Таким образом выполняется одна из задач, номер которой 5, 6, 7 или 9. Задания делаются по образцу из примеров и оформляются в качестве отчётов. Перед сдачей необходимо внимательно проверить правильность выполнения, чтобы избежать понижения баллов.

Срок выполнения лабораторной работы с сохранением баллов – 2 недели.

Используемые языки: C, C++, C#, Java, Python

4.3. Метрики Лоренца и Кидда

4.3.1. Теоретические сведения

Комплексный набор метрик, предложенный в 1994 г. М. Лоренцем и Д. Киддом (M. Lorenz и J. Kidd), имеет практическую направленность на использование в промышленной разработке программного обеспечения.

Набор включает 10 метрик, которые, в свою очередь, классифицируют в следующие группы:

- метрики размера, основанные на подсчете свойств и операций для отдельных классов, а также их средних значений для всей объектно-ориентированной системы;
- метрики наследования, учитывающие способы повторного использования операций в иерархии классов;
- внутренние метрики, отвечающие на вопросы связности и кодирования;
- внешние метрики, изучающие сцепление и повторное использование.

Размер класса CS (Class Size) – общий размер класса определяется на основании определения следующих показателей:

- общее количество операций;
- количество свойств.

Указанные измерения в обоих случаях следует проводить с учетом частных (приватных) и наследуемых экземплярных операций, которые инкапсулируются внутри класса:

$$CS = C_E + S_E, \quad (4.3.1)$$

где C_E – количество инкапсулированных классом методов (операций); S_E – количество инкапсулированных классом свойств.

CS может определяться взвешенной суммой C_E и S_E . Следует заметить, что метрика WMC Чидамбера и Кемерера также является взвешенной метрикой размера класса.

Большие значения метрики CS указывают на то, что класс имеет слишком много обязанностей, что уменьшает возможность повторного использования класса, усложняет его реализацию и тестирование.

При определении размера класса больший удельный вес придают унаследованным (публичным) операциям и свойствам, потому что

приватные операции и свойства обеспечивают специализацию и являются более локализованными в проекте.

Могут вычисляться средние количества свойств и операций класса. Чем меньше среднее значение размера CS , тем больше вероятность повторного использования класса. Рекомендуемое значение метрики CS ограничено сверху 20 инкапсулированными методами и свойствами ($CS \leq 20$).

Количество операций, переопределяемых подклассом NOO (Number of Operations Overridden by a Subclass). Переопределением называют случай, когда подкласс замещает операцию, унаследованную от суперкласса, своей собственной версией.

Большие значения NOO указывают на возникшие проблемы проектирования. Понятно, что подкласс должен расширять операции суперкласса, что проявляется в виде новых имен операций. Если же значение метрики NOO достаточно велико, то это означает нарушение разработчиком абстракции суперкласса. Это явление ослабляет иерархию классов, усложняет тестирование и модификацию программного обеспечения. Рекомендуемое значение метрики NOO составляет три метода.

Количество операций, добавленных подклассом NOA (Number of Operations Added by a Subclass), определяется количеством добавленных относительно родительского класса собственных методов (операций):

$$NOA = N_{\Sigma}, \quad (4.3.2)$$

где N_{Σ} – количество новых методов класса, добавленных относительно суперкласса.

С увеличением NOA подкласс приобретает меньшую общность со своим суперклассом, что требует больших трудозатрат по тестированию и внесению изменений. При увеличении высоты дерева иерархии классов (с ростом значения DIT) должно уменьшаться количество новых методов классов нижних уровней. Для рекомендуемых граничных значений размера класса ($CS = 20$) и высоты дерева иерархии классов ($DIT = 6$) значение NOA ограничено значением 4 ($NOA \leq 4$).

Индекс специализации SI (Specialization Index) характеризует грубую оценку степени специализации каждого подкласса при добавлении, удалении или переопределении операций:

$$SI = \frac{NOO \cdot u}{M_{\text{общ}}}, \quad (4.3.3)$$

где u – номер уровня в иерархии, на котором находится подкласс;
 $M_{\text{общ}}$ – общее количество методов класса.

Чем выше значение метрики SI , тем выше вероятность того, что в иерархии классов есть отдельные экземпляры, нарушающие абстракцию суперкласса. Рекомендуемое значение показателя SI ограничено сверху величиной 0,15, т. е. $SI \leq 0,15$.

Следующая группа метрик предназначена для оценки операций в классах. Как правило, методы бывают небольшими как по размеру, так и по логической сложности. В то же время истинные характеристики операций помогают более глубоко понять особенности создаваемой системы.

Средний размер операции AOS (Average Operation Size) определяется количеством сообщений, порождаемых операцией. В качестве оценки размера может использоваться количество строк программы, однако *LOC*-оценки приводят к известным проблемам. Иным вариантом может являться количество сообщений, посланных операцией.

Рост значения данного показателя означает, что обязанности размещены в классе не очень удачно. Рекомендуемое значение метрики *AOS* не должно превышать 9. Увеличение среднего размера относительно этой границы рассматривают как показатель неудачного проектирования обязанностей класса.

Сложность операции OC (Operation Complexity) может быть вычислена на основе стандартных метрик сложности (например, с помощью *LOC*- или *FP*-оценок, метрики цикломатической сложности, метрики Холстеда). Лоренц и Кидд предложили вычислять значение *OC* суммированием оценок с весовыми коэффициентами, приведенными в табл. 4.1.

Рекомендуемое значение метрики ограничено числом 65 ($OC \leq 65$).

Таблица 4.1. Весовые коэффициенты действий

Действие	Вес
Определение (описание) переменной-параметра	0,3
Определение (описание) временной переменной	0,5
Присваивание значения	0,5
Вложенное выражение	0,5
Сообщение без параметров	1
Арифметическая операция	2
Сообщение с параметрами	3
Вызов стандартной функции интерфейса (<i>API</i>)	5
Вызов пользовательской функции (простой вызов)	7

Среднее количество параметров на операцию *ANP* (Average Number of Parameters per operation) определяется отношением числа параметров к количеству операций (методов) класса.

$$ANP = \frac{\text{Количество параметров}}{\text{Количество операций}} \quad (4.3.4)$$

Чем больше параметров у операции, тем сложнее взаимодействие между объектами. Поэтому значение метрики *ANP* должно быть как можно меньшим. Рекомендуемое значение $ANP = 0,7$.

Следующая группа метрик предназначена для оценки показателей процесса разработки ПС. Центральным вопросом в процессе разработки является прогноз размера создаваемого продукта.

Количество описаний сценариев NSS (Number of Scenario Scripts) измеряется или количеством классов, реализующих требования к ПО, или количеством состояний для каждого класса, или количеством методов класса. При своем не совсем обычном способе измерения метрика *NSS* является достаточно эффективным индикатором размера создаваемой программы. Рекомендуется не менее одного сценария. Рост количества сценариев неминуемо ведет к увеличению размера программы.

Количество ключевых классов NKC (Number of Key Classes) адекватно характеризует предстоящий объем работы по программированию. Ключевой класс прямо связан с проблемной областью, для которой предназначена система, поэтому, если предметная область специфична, то возможность повторного использования существующего ключевого класса маловероятна и требуется самостоятельная разработка «с нуля». Авторы этого набора метрик полагают, что в типовой объектно-ориентированной системе на долю ключевых классов приходится от 20 до 40% общего количества классов. Остальные классы реализуют общую инфраструктуру (интерфейсы, коммуникации, базы данных). Рекомендуется ограничивать значение метрики снизу значением 0,2. Если значение метрики $NKC < 0,2$ от общего количества классов системы, следует пересмотреть выделение классов.

Количество подсистем NSUB (Number of subsystem) определяется непосредственным подсчетом. Количество подсистем обеспечивает понимание таких вопросов, как размещение ресурсов, планирование (с акцентом на параллельной разработке), общие затраты на интеграцию. Рекомендуется выделять в программном комплексе (системе)

не менее трех подсистем (т. е. значение $NSUB > 3$). Количество подсистем характеризует трудоемкость и управляемость проекта.

Значения метрик последней группы целесообразно накапливать по мере выполнения проектов по разработке ПС. Эти данные могут применяться для вычисления показателей производительности. Совместное применение метрик позволяет оценивать необходимые ресурсы на выполнение проекта: затраты, продолжительность разработки, численность привлекаемого персонала и другие характеристики.

4.3.2. Задача «Платеж за электроэнергию»

Необходимо определить класс, описывающий платеж за электроэнергию. В рамках класса следует предусмотреть следующие поля:

- фамилия плательщика;
- потребление электроэнергии за оплачиваемый месяц;
- нормативное среднemesячное потребление;
- тариф (стоимость одного киловатт-часа).

Рекомендуется применять следующие методы:

- вычисление суммы оплаты;
- формирование сводной информации по одному платежу (вид платежа, фамилия, сумма, потребление).

Платеж может выполняться по показаниям счетчика или по нормативно установленному среднemesячному потреблению. В процессе эксплуатации программы тариф и нормативно установленное среднemesячное потребление не изменяются. Для создания конкретного платежа предусмотреть соответствующий конструктор.

Все платежи должны сохраняться в архиве. Запросы по ведению архива выполняются статическими методами класса «Запрос»:

- занесение платежей в архив. Данные платежа вводятся с клавиатуры. Ввод отрицательного показания счетчика означает оплату по нормативно установленному среднemesячному потреблению;
- вывод сводной информации из архива.

В основном классе «Платежи» следует сформировать архив платежей. Архив моделируется массивом объектов. По данным архива предусмотреть выдачу сводной информации о платежах.

При решении задачи необходимо разработать исходный код программы, а также определить оценки характеристик программы на основе объектно-ориентированных метрик Лоренца и Кидда.

Текст программы на языке C# для реализации возможного алгоритма решения поставленной задачи представлен на рис. 4.1.

Номера строк	Строки программы
1	using System;
2	using System.Collections.Generic;
3	using System.Text;
4	namespace EX1
5	{
6	class Электро
7	{
8	private static double тариф = 1.84;
9	private static int потреблениеСреднее = 300;
10	private string фамилия;
11	private int потребление;
12	public Электро(string фамилия)
13	{
14	this.фамилия = фамилия;
15	потребление = потреблениеСреднее;
16	}
17	public Электро(string фамилия, int текущее, int предыдущее)


```
18 {
19     this.фамилия = фамилия;
20     потребление = текущее - предыдущее;
21 }
22 public double Сумма()
23 {
24     return потребление * тариф;
25 }
26 public string Инфо()
27 {
28     return string.Format("{0,-20}{1,-20}{2,10:f2}{3,10:d6}",
29         "Электроэнергия", фамилия, Сумма(), потребление);
30 }
31 }
32 class Запрос
33 {
34     public static void Заполнить(Электро[] платеж)
35     {
36         string фам;
37         int счПред=0, счТек=0;
38         Console.Clear();
39         for (int i = 0; i < платеж.Length; i++)
40         {
41             Console.Write("Платеж " + i + ": Фамилия -> ");
42             фам = Console.ReadLine();
43             Console.Write("Платеж " + i + ": Текущее значение счетчика -> ");
44             счТек = int.Parse(Console.ReadLine());
45             if (счТек > 0)
46             {
47                 Console.Write("Платеж " + i + ": Предыдущее значение счетчика -> ");
48                 счПред = int.Parse(Console.ReadLine());
49             }
50             if (счТек <= 0)
51                 платеж[i] = new Электро(фам);
52             else
53                 платеж[i] = new Электро(фам, счТек, счПред);
54         }
55     }
56     public static void Вывести(Электро[] платеж)
57     {
58         for (int i = 0; i < платеж.Length; i++)
59             Console.WriteLine(платеж[i].Инфо());
60     }
61 }
62 class Платежи
63 {
```

```

64 static void Main(string[] args)
65 {
66     ConsoleKeyInfo rep;
67     Электро[] плэ;
68     int кпэ;
69     do
70     {
71         Console.Clear();
72         Console.Write("Количество платежей за электроэнергию: ");
73         кпэ = int.Parse(Console.ReadLine());
74         плэ = new Электро[кпэ];
75         Запрос.Заполнить(плэ);
76         Запрос.Вывести(плэ);
77         Console.WriteLine("Для выхода нажмите ESC");
78         rep = Console.ReadKey(true);
79     } while(rep.Key != ConsoleKey.Escape);
80     }
81     }
82     }

```

Рис. 4.1. Пример реализации программы «Платеж за электроэнергию»

Реализация программы

Текст программы на языке C# для реализации возможного алгоритма решения поставленной задачи представлен на рис. 4.1.

Оценка характеристик программы

Метрика размера класса CS определяется на основании следующих показателей:

- общее количество операций;
- количество свойств,

а значение рассчитывается по формуле

$$CS = C_{\Sigma} + S_{\Sigma},$$

где C_{Σ} – количество инкапсулированных классом методов; S_{Σ} – количество инкапсулируемых классом свойств (полей классов).

Для класса *Электро* число инкапсулированных методов $C_{\Sigma} = 4$, количество инкапсулированных полей $S_{\Sigma} = 4$ (см. рис. 4.1):

- *private static double тариф* = 1.84;
- *private static int потреблениеСреднее* = 300;
- *private string фамилия*;
- *private int потребление*.

Таким образом, для класса *Электро*

$$CS = C_{\Sigma} + S_{\Sigma} = 4 + 4 = 8.$$

Для класса *Запрос* количество инкапсулируемых методов $C_{\Sigma} = 2$, количество инкапсулированных полей $S_{\Sigma} = 0$, так как в классе не определены поля (см. рис. 4.1):

$$CS = C_{\Sigma} + S_{\Sigma} = 2 + 0 = 2.$$

Для класса *Платежи* количество инкапсулируемых методов $C_{\Sigma} = 1$, количество инкапсулированных полей $S_{\Sigma} = 0$, так как в классе не определены поля (см. рис. 4.1).

$$CS = C_{\Sigma} + S_{\Sigma} = 1 + 0 = 1.$$

Полученные значения метрик классов не превышают значения 20, следовательно, сложность классов не превышает требуемый уровень и разбиение программы на дополнительные модули не требуется.

Количество операций, переопределяемых подклассом *NOO*, количество операций, добавленных подклассом *NOA*, индекс специали-

зации *SI* для данного решения не определяются, так как в исходном коде программы не реализован принцип наследования.

Метрика среднего размера операции *AOS* определяется количеством сообщений, порождаемых операцией. В качестве посылаемого методом сообщения будем рассматривать обращение к методам других классов. Определим значение метрики для классов. В классе *Электро* обращения к методам осуществляются только в методе *public string Инфо()* (см. рис. 4.1, строка 29). Количество посылаемых сообщений равно 2:

- обращение к методу *string.Format()*;
- обращение к методу *Сумма()*.

Таким образом, для класса *Электро* $AOS = 2$.

В классе *Запрос* обращений к методам наблюдается 12 раз (см. определение метрики *СВО(Запрос)*). Для класса *Запрос* $AOS = 12$.

В классе *Платежи* встречается 10 обращений к методам (из определения метрики *СВО(Платежи)*), следовательно, для этого класса $AOS = 10$. Из полученных результатов следует, что наиболее сложным является класс *Запрос*.

Определим сложность операции *ОС*, которую будем вычислять по количеству строк в коде метода. При вычислении воспользуемся весовыми коэффициентами, предложенными Лоренцом и Киддом (табл. 4.2).

Таблица 4.2. Весовые коэффициенты действий

Действие	Вес
Определение (описание) переменной-параметра	0,3
Определение (описание) временной переменной	0,5
Присваивание значения	0,5
Вложенное выражение	0,5
Сообщение без параметров	1
Арифметическая операция	2
Сообщение с параметрами	3
Вызов стандартной функции интерфейса (<i>API</i>)	5
Вызов пользовательской функции (простой вызов)	7

Значения весовых коэффициентов зависят от количества резервируемой памяти, необходимой для размещения исполняемых кодов операций программы. Наименьшее количество кодовых операций требуют операторы описания параметров методов. Операторы описания переменных и массивов, а также операторы присваивания тре-

буют большего количества машинных кодов. Арифметические операции, операции обмена данными требуют еще большего объема памяти, так как кроме исполняемых машинных кодов требуется дополнительная память и для обрабатываемых данных, которые необходимо передавать из одной функции обработки в другую.

Определим количество строк по предложенным разновидностям действий для класса *Электро*, которые для удобства сведем в таблицы, рассматривая применяемые методы.

Таблица 4.3. Метод *public Электро(string фамилия)* (см. рис. 4.1, строки 12–16)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	2
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	0
Вызов пользовательской функции (простой вызов)	7	0

Расчет сложности операции *OC* дает следующий результат:

$$OC = 0,5 \cdot 2 = 1.$$

Таблица 4.4. Метод *public Электро(string фамилия, int текущее, int предыдущее)* (см. рис. 4.1, строки 17–21)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	2
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	1
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	0
Вызов пользовательской функции (простой вызов)	7	0

Значение метрики сложности операции $OC = 0,5 \cdot 2 + 2 \cdot 1 = 3$.

Таблица 4.5. Метод *public double Сумма()* (см. рис. 4.1, строки 22–25)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	0
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	1
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	0
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода $OC = 2 \cdot 1 = 2$.

Таблица 4.6. Метод *public string Инфо()* (см. рис. 4.1, строки 26–31)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	0
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	1
Вызов пользовательской функции (простой вызов)	7	1

Метрика сложности операции для данного метода:

$$OC = 5 \cdot 1 + 7 \cdot 1 = 12.$$

Из полученных значений метрики сложности операции можно сделать вывод, что все операции класса *Электро* имеют сложность низкого уровня, которая не превышает критическое значение (менее 65).

Определим показатели сложности операции OC для класса *Запрос*.

Таблица 4.7. Метод *public static void Заполнить(Электро[] платеж)* (см. рис. 4.1, строки 34–55)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	1
Определение (описание) временной переменной	0,5	4
Присваивание значения	0,5	8
Вложенное выражение	0,5	0
Сообщение без параметров	1	0

Окончание табл. 4.7

Действие	Вес	Количество строк
Арифметическая операция	2	4
Сообщение с параметрами	3	3
Вызов стандартной функции интерфейса (API)	5	10
Вызов пользовательской функции (простой вызов)	7	2

Метрика сложности операции для данного метода:

$$OC = 0,3 \cdot 1 + 0,5 \cdot 3 + 0,5 \cdot 8 + 2 \cdot 4 + 3 \cdot 3 + 5 \cdot 10 + 7 \cdot 2 = 87,3.$$

Таблица 4.8. Метод `public static void Вывести(Электро[] платеж)` (см. рис. 4.1, строки 56–60)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	1
Определение (описание) временной переменной	0,5	1
Присваивание значения	0,5	1
Вложенное выражение	0,5	0
Сообщение без параметров	1	1
Арифметическая операция	2	1
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	1
Вызов пользовательской функции (простой вызов)	7	1

Метрика сложности операции определится следующим образом:

$$OC = 0,3 \cdot 1 + 0,5 \cdot 1 + 0,5 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 5 \cdot 1 + 7 \cdot 1 = 16,3.$$

Из полученных значений видно, что одна из операций класса *Запрос* имеет значительный уровень сложности (значение метрики *OC* превышает 65).

Определим характеристику *OC* для класса *Платежи*.

Таблица 4.9. Метод `static void Main(string[] args)` (см. рис. 4.1, строки 64–81)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	1
Определение (описание) временной переменной	0,5	3
Присваивание значения	0,5	3
Вложенное выражение	0,5	1
Сообщение без параметров	1	2
Арифметическая операция	2	0
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	6
Вызов пользовательской функции (простой вызов)	7	2

Метрика сложности операции для данного метода:

$$OC = 0,3 \cdot 1 + 0,5 \cdot 3 + 0,5 \cdot 3 + 0,5 \cdot 1 + 1 \cdot 2 + 5 \cdot 6 + 7 \cdot 2 = 49,8.$$

В связи с тем что сложность операции класса *Платежи* не превышает 65, можно сделать вывод о допустимом уровне сложности данного метода.

Определим среднее количество параметров на операцию *ANP* (*Average Number of Parameters per operation*), которое рассчитывается по следующему соотношению:

$$ANP = \frac{\text{Количество_параметров}}{\text{Количество_операций}}$$

Количество параметров в классе *Электро* четыре: один параметр в методе *public Электро(string фамилия)*, три параметра в методе *public Электро(string фамилия, int текущее, int предыдущее)*, остальные методы входных параметров не имеют. Количество методов в классе четыре, следовательно,

$$ANP = 4 / 4 = 1.$$

Количество параметров в классе *Запрос* два: один – в методе *public static void Заполнить(Электро[] платеж)*, другой – в методе *public static void Вывести(Электро[] платеж)*. Количество методов – два, отсюда следует, что

$$ANP = 2 / 2 = 1.$$

Параметры в классе *Платежи* отсутствуют, в классе имеется один метод, поэтому

$$ANP = 0 / 2 = 0.$$

Среднее количество параметров на операцию в большинстве классов выше допустимого (более 0,7), что говорит о низком качестве программы.

Определим количество сценариев *NSS* (*Number of Scenario Scripts*), опираясь на количество методов в классе. Для класса *Электро* значение *NSS* = 4, для класса *Запрос* *NSS* = 2, для класса *Платежи* показатель *NSS* = 1.

Количество сценариев для исследуемой программы невелико, что объясняет сравнительно небольшие размеры программы.

Количество ключевых классов *NKC* (*Number of Key Classes*) в рассматриваемой программе равно трем: Все классы, определенные в

программе, можно назвать ключевыми, так как они напрямую связаны с проблемной областью поставленной задачи. Таким образом, $NKC = 1$, т. е. количество ключевых классов от общего количества классов программы составляет 100%.

В рассматриваемом решении можно выделить три подсистемы:

- класс, определяющий элемент платежа как объект;
- класс, определяющий операции создания объектов и вывода информации по ним;
- основной управляющий класс.

Таким образом, количество подсистем $NSUB$ (*Number of subsystem*) = 3. Количество подсистем программы обеспечивает достаточную управляемость проекта и невысокую трудоемкость.

4.3.3. Задача «Геометрия окружности и прямоугольника»

В предметной области «Геометрия» определены информационные объекты, описывающие понятия «Точка», «Окружность» и «Прямоугольник» в виде классов. Классы «Окружность» и «Прямоугольник» являются наследниками класса «Точка». Каждый из классов имеет метод вывода параметров объекта. Следует иметь в виду, что метод вывода в базовом классе не виртуальный и не абстрактный.

Методы вывода в наследниках переопределены. Вызываемый метод будет определяться типом ссылки.

Базовый класс «Точка» описан следующим образом:

- закрытые поля:
 - координата по оси X (вещественная);
 - координата по оси Y (вещественная);
- метод: вывод координат точки.

Класс «Окружность» задан следующими элементами:

- закрытые поля:
 - координаты центра – унаследованные поля;
 - радиус (вещественный);
- метод: вывод координат центра и длины окружности.

Класс «Прямоугольник» задается так:

- закрытые поля:
 - координаты левого верхнего угла – унаследованные поля;

- основание (вещественное);
- высота (вещественная);

• метод: вывод координат левого верхнего угла и площади.

В основном классе необходимо сформировать список геометрических объектов разных типов в виде массива и вывести параметры объектов на экран. Разработать исходный код программы, по которому определить оценки характеристик программы на основе объектно-ориентированных метрик Лоренца и Киддэ.

Номера строк	Строки программы
1	namespace EX1_1
2	{
3	class Точка
4	{
5	protected double x, y;
6	public Точка(double x, double y)
7	{
8	this.x = x; this.y = y;
9	}
10	public void Показать()
11	{
12	Console.WriteLine("Точка ({0:f2}:{1:f2})", x, y);
13	}
14	}
15	class Окружность: Точка
16	{
17	private double r;
18	public Окружность(double x, double y, double r): base(x, y)
19	{
20	this.r = r;
21	}
22	public new void Показать()
23	{
24	Console.WriteLine("Центр({0:f2}:{1:f2}) Длина={2:f2}", x, y, 2*Math.PI*r);
25	}
26	}
27	
28	class Прямоугольник: Точка
29	{
30	private double a, h;

```

31 public Прямоугольник(double x, double y, double a, double h): base(x, y)
32 {
33     this.a = a; this.h = h;
34 }
35 public new void Показать()
36 {
37     Console.WriteLine("Угол({0:f2}:{1:f2}) Площадь={2:f2}", x, y, a*h);
38 }
39 }
40 class Program
41 {
42     static void Main(string[] args)
43     {
44         Точка[] список = new Точка[6];
45         список[0] = new Точка(-1.0, -1.0);
46         список[1] = new Окружность(1.0, 1.0, 1.0);
47         список[2] = new Прямоугольник(10.0, 10.0, 10.0, 10.0);
48         список[3] = new Точка(-2.0, -2.0);
49         список[4] = new Окружность(3.0, 3.0, 3.0);
50         список[5] = new Прямоугольник(20.0, 20.0, 0.5, 0.5);
51         for (int i = 0; i < список.Length; i++) список[i].Показать();
52         Console.WriteLine();
53         for (int i = 0; i < список.Length; i++)
54         {
55             if (список[i] is Точка)
56                 ((Точка)список[i]).Показать();
57             else
58                 if (список[i] is Окружность)
59                     ((Окружность)список[i]).Показать();
60                 else
61                     if (список[i] is Прямоугольник)
62                         ((Прямоугольник)список[i]).Показать();
63         }
64         Console.WriteLine();
65         for (int i = 0; i < список.Length; i++)
66             switch (список[i].GetType().Name)
67             {
68                 case "Точка": ((Точка)список[i]).Показать(); break;
69                 case "Окружность": ((Окружность)список[i]).Показать(); break;
70                 case "Прямоугольник": ((Прямоугольник)список[i]).Показать(); break;
71             }
72     }
73 }
74 }

```

Рис. 4.2. Пример реализации программы «Геометрия»

реализация программы

Текст программы на языке C# для реализации возможного алгоритма решения поставленной задачи представлен на рис. 4.2.

оценка характеристик программы

Проанализируем текст программы для оценки ее качества с помощью метрик Лоренца и Кидда, которые позволяют оценить меру сложности объектно-ориентированной программы на основе анализа организационной структуры классов.

Исходный код программы включает четыре класса. К ним относятся следующие классы (см. рис. 4.2):

- *class Точка* – базовый класс, определяющий понятие точка, расположенная на координатной плоскости;
- *class Окружность* – производный класс от *class Точка*, определяющий понятие *окружность* с центром в точке, расположенной на координатной плоскости;
- *class Прямоугольник* – производный класс от *class Точка*, определяющий понятие *прямоугольник*, расположение которого на плоскости определяется координатами точки верхнего левого его угла;
- *class Program* – класс, в котором содержится основной метод обработки массивов объектов типа *Точка*, *Окружность* и *Прямоугольник*.

Согласно теории Лоренца и Кидда для оценки качества объектно-ориентированных программ определяются следующие метрики:

- размер класса *CS (Class Size)* – общий размер класса;
- количество операций, переопределяемых подклассом *NOO (Number of Operations Overridden by a Subclass)*;

- количество операций, добавленных подклассом *NOA* (*Number of Operations Added by a Subclass*);
- индекс специализации *SI* (*Specialization Index*);
- средний размер операции *AOS* (*Average Operation Size*);
- сложность операции *OC* (*Operation Complexity*);
- среднее количество параметров на операцию *ANP* (*Average Number of Parameters per operation*);
- количество описаний сценариев *NSS* (*Number of Scenario Scripts*);
- количество ключевых классов *NKC* (*Number of Key Classes*);
- количество подсистем *NSUB* (*Number of subsystem*).

Метрики размера класса *CS* исчисляются на основании определения общего количества операций и количества свойств:

$$CS = C_{\Sigma} + S_{\Sigma},$$

где C_{Σ} – количество инкапсулированных классом методов; S_{Σ} – количество инкапсулируемых классом свойств (полей классов).

Класс *Точка* включает два инкапсулированных поля x и y : *protected double x, y*. В состав класса входят два инкапсулированных метода: *public Точка(double x, double y)* и *public void Показать()*. Таким образом, $C_{\Sigma} = 2$, $S_{\Sigma} = 2$, $CS = C_{\Sigma} + S_{\Sigma} = 2 + 2 = 4$. Значение метрики находится в установленных пределах (менее 20), сложность класса – незначительная.

Класс *Окружность* включает одно инкапсулированное поле *private double r*. В состав класса входят два инкапсулированных метода: *public Окружность(double x, double y, double r): base(x, y)* и *public new void Показать()*.

Таким образом, $C_{\Sigma} = 2$, $S_{\Sigma} = 1$, $CS = C_{\Sigma} + S_{\Sigma} = 2 + 1 = 3$. Значение метрики находится в установленных пределах (менее 20), сложность класса – незначительная.

Класс *Прямоугольник* включает два инкапсулированных поля a и h : *private double a, h*. В состав класса входят два инкапсулированных метода: *public Прямоугольник(double x, double y, double a, double h): base(x, y)* и *public new void Показать()*.

Таким образом, $C_{\Sigma} = 2$, $S_{\Sigma} = 2$, $CS = C_{\Sigma} + S_{\Sigma} = 2 + 2 = 4$. Значение метрики в установленных пределах (менее 20), сложность класса – незначительная.

В состав класса *Program* входит только один метод *static void Main(string[] args)*, поля отсутствуют. Таким образом, $C_{\Sigma} = 1$, $S_{\Sigma} = 0$,

$CS = C_{\Sigma} + S_{\Sigma} = 1 + 0 = 1$. Значение метрики находится в установленных пределах (менее 20), сложность класса минимальна.

В программе реализован принцип наследования. Базовым классом является класс *Точка*, производными классами (наследниками) являются *Окружность* и *Прямоугольник*. Классы *Окружность* и *Прямоугольник* содержат по одному переопределяемому методу с одним и тем же именем *public new void Показать()*. Следовательно, $NOO(Окружность) = 1$; $NOO(Прямоугольник) = 1$.

Значения найденных характеристик не превышает критический уровень (менее 3). Иерархия классов – достаточно устойчивая, тестирование и модификация классов, как можно ожидать, будут несложными.

Показатель количества добавленных операций в классах *Окружность* и *Прямоугольник* определится следующим образом: $NOA(Окружность) = 1$; $NOA(Прямоугольник) = 1$.

Уровень трудозатрат по тестированию и внесению изменений будет невысоким.

Производные классы *Окружность* и *Прямоугольник* находятся на втором уровне иерархии, на первом расположен базовый класс *Точка*.

Следовательно, индекс специализации *SI* для каждого из классов будет иметь следующие значения:

$$SI(Окружность) = \frac{NOO \cdot u}{M_{\text{общ}}} = \frac{1 \cdot 2}{2} = 1;$$

$$SI(Прямоугольник) = \frac{NOO \cdot u}{M_{\text{общ}}} = \frac{1 \cdot 2}{2} = 1.$$

Индекс специализации производных классов достаточно высок (более 0,15). Вероятность нарушений абстракции базового класса отдельными объектами высока. Рекомендуется снижение количества переопределяемых методов в производных классах.

Определим средний размер операций *AOS* классов программы путем подсчета количества обращений к внешним методам. В классе *Точка* реализовано одно обращение к внешнему методу (см. рис. 4.1, строка 12): $AOS(Точка) = 1$.

В классе *Окружность* реализовано два обращения к методам классов *Console* и *Math* (см. рис. 4.1, строка 24): $AOS(Окружность) = 2$.

В классе *Прямоугольник* реализовано одно обращение к внешнему методу (см. рис. 4.1, строка 37): $AOS(Прямоугольник) = 1$.

В классе *Program* реализовано 17 обращений к методам других классов (см. рис. 4.1, строки 44–70): $AOS(Program) = 17$.

Из полученных значений показателей следует, что классы спроектированы достаточно удачно, так как вызовы методов всех классов главным образом сосредоточены в основном управляющем классе *Program*.

Рассмотрим сложность операций классов *OC* на основе *LOC*-оценок.

Показатели характеристик рассчитаем методом сложения оценок с весовыми коэффициентами, предложенными Лоренцом и Киддом.

Значения весовых коэффициентов зависят от количества резервируемой памяти, необходимой для размещения исполняемых кодов операций программы.

Наименьшее количество кодовых операций требуют операторы описания параметров методов. Операторы описания переменных и массивов, а также операторы присваивания требуют большего количества машинных кодов. Арифметические операции, операции обмена данными требуют еще большего объема памяти, т.к. помимо исполняемых машинных кодов требуется дополнительная память и для обрабатываемых данных, которые необходимо передавать из одной функции обработки в другую.

Определим количество строк по предложенным разновидностям действий для класса *Точка*, которые сведем в таблицы.

Таблица 4.10. Метод *public Точка(double x, double y)* (см. рис. 4.1, строки 6–9)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	2
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	2
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (<i>API</i>)	5	0
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода:

$$OC = 0,3 \cdot 2 + 0,5 \cdot 2 = 1,6.$$

Таблица 4.11. Метод *public void Показать()* (см. рис. 4.1, строки 10–14)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	0
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	1
Вызов стандартной функции интерфейса (API)	5	1
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода:

$$OC = 3 \cdot 1 + 5 \cdot 1 = 8.$$

Определим количество строк по предложенным разновидностям действий для класса *Окружность*, которые сведем в таблицы.

Таблица 4.12. Метод *public Окружность(double x, double y, double r): base(x, y)* (см. рис. 4.1, строки 18–21)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	3
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	1
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	0
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода:

$$OC = 0,3 \cdot 3 + 0,5 \cdot 1 = 1,4.$$

Таблица 4.13. Метод *public new void Показать()* (см. рис. 4.1, строки 22–25)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	0
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0

Окончание табл. 4.13

Действие	Вес	Количество строк
Сообщение с параметрами	3	1
Вызов стандартной функции интерфейса (API)	5	2
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода:

$$OC = 3 \cdot 1 + 5 \cdot 2 = 13.$$

Определим количество строк по предложенным разновидностям действий для класса *Прямоугольник*, которые сведем в таблицы.

Таблица 4.14. Метод *public Прямоугольник(double x, double y, double a, double h): base(x, y)* (см. рис. 4.1, строки 31–34)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	4
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	2
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	0
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода:

$$OC = 0,3 \cdot 4 + 0,5 \cdot 2 = 2,2.$$

Таблица 4.15. Метод *public new void Показать()* (см. рис. 4.1, строки 35–38)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	0
Определение (описание) временной переменной	0,5	0
Присваивание значения	0,5	0
Вложенное выражение	0,5	0
Сообщение без параметров	1	0
Арифметическая операция	2	0
Сообщение с параметрами	3	1
Вызов стандартной функции интерфейса (API)	5	1
Вызов пользовательской функции (простой вызов)	7	0

Метрика сложности операции для данного метода:

$$OC = 3 \cdot 1 + 5 \cdot 1 = 8.$$

Определим количество строк по предложенным разновидностям действий для класса *Прямоугольник*, которые сведем в таблицу.

Таблица 4.16. Метод *static void Main(string[] args)* (см. рис. 4.1, строки 42–72)

Действие	Вес	Количество строк
Определение (описание) переменной-параметра	0,3	1
Определение (описание) временной переменной	0,5	1
Присваивание значения	0,5	9
Вложенное выражение	0,5	0
Сообщение без параметров	1	2
Арифметическая операция	2	3
Сообщение с параметрами	3	0
Вызов стандартной функции интерфейса (API)	5	2
Вызов пользовательской функции (простой вызов)	7	12

Метрика сложности операции для данного метода определится следующим образом:

$$OC = 0,3 \cdot 1 + 0,5 \cdot 1 + 0,5 \cdot 9 + 1 \cdot 2 + 2 \cdot 3 + 5 \cdot 2 + 7 \cdot 12 = 107,3.$$

Сложность операций классов *Точка*, *Окружность* и *Прямоугольник* находится в установленных пределах (менее 65). Сложность основного класса *Program* превышает эту норму (более 65). Основным класс целесообразно разбить на два и более класса, т. е. часть операций рекомендуется перенести в дополнительный класс.

Рассчитаем количество параметров на операцию класса *ANP*. Для класса *Точка* (см. рис. 4.1, строки 3 – 14):

$$ANP = \frac{\text{Количество_параметров}}{\text{Количество_операций}} = \frac{2}{2} = 1.$$

Для класса *Окружность*:

$$ANP = \frac{\text{Количество_параметров}}{\text{Количество_операций}} = \frac{3}{2} = 1,5.$$

Для класса *Прямоугольник*:

$$ANP = \frac{\text{Количество_параметров}}{\text{Количество_операций}} = \frac{4}{2} = 2.$$

Для класса *Program*:

$$ANP = \frac{\text{Количество_параметров}}{\text{Количество_операций}} = \frac{1}{1} = 1.$$

Все операции классов программы имеют высокие значения характеристик по количеству используемых параметров в объектах (более 0,7), что значительно усложняет взаимодействие между объектами.

Количество сценариев классов NSS определим по количеству используемых методов в классах. В классе *Точка* определены два метода, следовательно, $NSS = 2$. Для классов *Окружность* и *Прямоугольник* этот показатель будет $NSS = 2$, так как в них используется два метода.

В классе *Program* реализован один метод, следовательно, $NSS = 1$. Количество сценариев программы сравнительно невелико, что определяет небольшие размеры программы.

Количество ключевых классов программы $NKC = 1$ (100%), поскольку все классы программы непосредственно связаны с проблемной областью вычисления местоположения геометрических фигур, определяемого координатой одной характерной точки.

В программе можно выделить три подсистемы:

- подсистему определения объекта – окружность;
- подсистему определения объекта – прямоугольник;
- подсистему обработки информации по геометрическим фигурам.

Таким образом, $NSUB = 3$. При таком значении трудоемкость и управляемость проектом можно считать достаточно низкими.

4.3.4. Задачи для самостоятельного решения

В предлагаемых задачах необходимо разработать программу, реализующую решение по приведенному условию, а затем оценить характеристики разработанной программы на основе применения объектно-ориентированных метрик Лоренца и Кидда.

Задача 1. Определить понятие «Автомобиль». Состояние объекта определяется полями:

- регистрационный номер (строка из 6 символов);
- код региона России (целое число);
- фамилия владельца (строка до 20 символов);
- номер паспорта владельца (длинное целое число).

В таблице регистрации автомобилей удалить данные обо всех автомобилях, принадлежащих владельцу с заданным номером паспорта.

Задача 2. Определить понятие «Автомобиль». Состояние объекта определяется полями:

- регистрационный номер (строка из 6 символов);
- код региона России (целое число);
- номер паспорта владельца (длинное целое число).

Регистрационный номер образован по шаблону «БЦЦЦББ», где символ Б – буква, Ц – цифра (например, «К233ВО»). По таблице регистрации вычислить количество автомобилей в заданном регионе России, цифровая группа номера которых 767.

Задача 3. Определить понятие «Автомобиль». Состояние объекта определяется полями:

- регистрационный номер (строка из 6 символов);
- код региона России (целое число);
- номер паспорта владельца (длинное целое число).

Регистрационный номер образован по шаблону: «БЦЦЦББ», где символ Б – буква, Ц – цифра (например, «К233ВО»). По таблице регистрации вычислить количество автомобилей, у которых сумма цифр номера равна 27.

Задача 4. Определить понятие «Автомобиль». Состояние объекта определяется полями:

- регистрационный номер (строка из 6 символов);
- код региона России (целое число);
- номер паспорта владельца (длинное целое число).

Регистрационный номер образован по шаблону: «БЦЦЦББ», где символ Б – буква, Ц – цифра (например, «К233ВО»). По данным таблицы автомобилей вывести характеристики всех автомобилей, у которых цифры номера образуют неубывающую последовательность.

Задача 5. Определить понятие «Абонент телефонной сети». Состояние объекта определяется полями:

- номер телефона (семизначное целое число);
- фамилия (строка до 20 символов);
- номер паспорта (длинное целое число).

Базируясь на указанном понятии, определить понятие «Задолженность», дополнительно определив понятие «Абонент телефонной сети» полем «Долг по оплате» (вещественное число). Определить и вывести данные абонента, имеющего максимальный долг по оплате среди всех абонентов.

Задача 6. Определить понятие «Абонент телефонной сети». Состояние объекта определяется полями:

- номер телефона (семизначное целое число);
- фамилия (строка до 20 символов);
- номер паспорта (длинное целое число).

Базируясь на указанном понятии, определить понятие «Задолженность», дополнительно определив понятие «Абонент телефонной сети» полем «Долг по оплате» (вещественное число). По таблице абонентов вывести номера телефонов и фамилии абонентов, у которых долг по оплате превышает заданный предел.

Задача 7. Определить понятие «Человек». Состояние объекта определяется полями:

- номер паспорта (семизначное целое число);
- фамилия (строка до 20 символов).

Базируясь на этом понятии, определить понятие «Проживающий», дополнительно определив понятие «Человек» полями «Наименование отеля» (строка до 15 символов) и «Количество суток проживания» (целое число). Вывести фамилии всех лиц, проживших в отеле «Загородный» наибольшее количество суток.

Задача 8. Определить понятие «Автомобиль». Состояние объекта определяется полями:

- регистрационный номер (строка из 6 символов);
- код региона России (целое число);
- фамилия владельца (строка до 20 символов).

По таблице регистрации автомобилей вычислить и вывести количество автомобилей, зарегистрированных на заданную фамилию.

Задача 9. Определить понятие «Человек». Состояние объекта определяется полями:

- номер паспорта (семизначное целое число);
- фамилия (строка до 20 символов).

Базируясь на указанном понятии, определить понятие «Проживающий», дополнительно определив понятие «Человек» полями «Наименование отеля» (строка до 15 символов) и «Количество суток проживания» (целое число). По таблице проживающих вычислить количество суток, которые прожил гость с фамилией «Петров» в заданном отеле.

